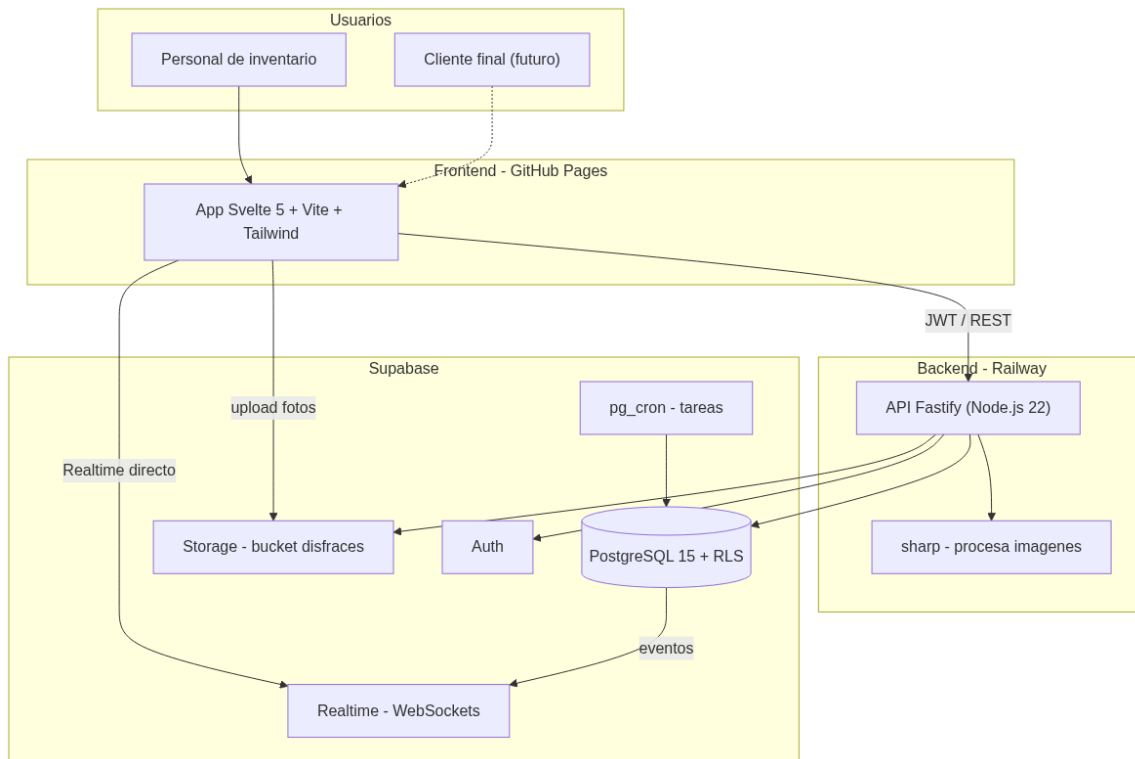


SISTEMA DE INVENTARIO DE DISFRACES

Documentación técnica y funcional del proyecto
Arquitectura Web App + Supabase + Node.js + Svelte



Proyecto: Sistema de Inventario de Disfraces

Versión: 1.0

Fecha: Octubre 2026

Repositorio: <https://github.com/disfracesrosaelvira/productos>

Índice

Índice	2
1. Introducción	4
1.1 Contexto	4
1.2 Problema	4
1.3 Justificación	4
2. Objetivos.....	4
2.1 Objetivo general	4
2.2 Objetivos específicos	4
3. Alcances y limitaciones.....	4
3.1 Alcances	4
3.2 Limitaciones / fuera de alcance.....	5
4. Stack tecnológico.....	5
5. Arquitectura del sistema	5
5.1 Arquitectura de alto nivel.....	5
5.2 Diagrama de componentes (bajo nivel)	6
6. Diagramas de flujo.....	7
6.1 Flujo de subida de fotografías	7
6.2 Flujo de stock en tiempo real	8
7. Criterios de trazabilidad y lógica de negocio.....	9
7.1 Identificadores.....	9
7.2 Generación automática de SKU.....	9
7.3 Lógica condicional (switch / categorización).....	9
7.4 Triggers de base de datos (trazabilidad automática)	10
7.5 Matriz de trazabilidad.....	10
8. Modelo de datos.....	11
8.1 Diagrama Entidad-Relación	11
8.2 Diccionario de tablas	12
9. Backend (Node.js + Fastify)	12
9.1 Estructura del proyecto	12
9.2 Endpoints.....	12
9.3 Funciones principales	13

10. Frontend (Svelte 5 + Vite + Tailwind)	14
10.1 Estructura	14
10.2 Rutas y vistas	14
10.3 Componentes clave	14
11. Mockups	15
11.1 Baja fidelidad (wireframes)	15
11.2 Alta fidelidad	19
12. Seguridad	23
13. Despliegue	23
14. Manual de uso	24
15. Anexos	24
Anexo A. Guion SQL de la base de datos.....	24
Anexo B. Script de carga masiva (Python).....	31
Anexo C. Código Mermaid de los diagramas.....	31
Anexo D. Glosario	34
Anexo E. Repositorio y recursos	34

1. Introducción

1.1 Contexto

El negocio de alquiler y venta de disfraces maneja un catálogo amplio (más de 2,000 fotografías) organizado por categorías temáticas (superhéroes, profesiones, series, películas, Halloween), géneros (niño, niña, hombre, mujer, unisex), tallas y calidades de tela. El control de este inventario se realizaba de forma manual, lo que dificultaba conocer el stock real, registrar ventas y mantener las fotografías al día.

1.2 Problema

No existía una herramienta centralizada que permitiera consultar el inventario en tiempo real, subir fotos desde el móvil, controlar stock automáticamente al vender y obtener reportes. Esto generaba errores de stock, pérdida de tiempo y falta de trazabilidad de las operaciones.

1.3 Justificación

Se implementa una aplicación web con base de datos en la nube (Supabase), backend Node.js y frontend Svelte. La solución es de bajo costo (planes gratuitos), escalable a un módulo de ventas y alquiler, y permite el trabajo colaborativo con actualización en tiempo real del stock.

2. Objetivos

2.1 Objetivo general

Desarrollar un sistema web de control de inventario de disfraces que gestione productos, variantes (talla, género, tela) y stock en tiempo real, con carga de fotografías desde el móvil y registro de ventas.

2.2 Objetivos específicos

- Centralizar el catálogo de disfraces con fotografías, categorías, tallas y géneros.
- Automatizar el descuento de stock al registrar una venta mediante disparadores (triggers).
- Permitir la subida de fotografías desde el móvil con compresión automática.
- Ofrecer filtros de búsqueda por nombre, categoría y género.
- Generar reportes de bajo stock, resumen de ventas y productos más vendidos.
- Actualizar la interfaz en tiempo real (Realtime) cuando cambia el stock.
- Desplegar el sistema en la nube con costo cero (GitHub Pages, Railway, Supabase).

3. Alcances y limitaciones

3.1 Alcances

- Gestión de productos, categorías, variantes, imágenes y ventas.
- Autenticación de usuarios y control de acceso por RLS en la base de datos.

- Carga masiva inicial de fotografías y metadatos con Python.
- Interfaz responsive (móvil y escritorio) con Tailwind CSS.
- Reportes operativos y actualización en vivo del inventario.

3.2 Limitaciones / fuera de alcance

- Pasarela de pagos en línea (las ventas se registran manualmente).
- Módulo de alquiler (previsto como extensión futura).
- Tienda pública para clientes finales (el sistema es de uso interno).
- Notificaciones por correo o WhatsApp.

4. Stack tecnológico

Capa	Tecnología	Propósito
Frontend	Svelte 5 + Vite + Tailwind CSS 4	Interfaz de usuario
Backend	Node.js 22 + Fastify 5	API REST y lógica de negocio
Base de datos	Supabase PostgreSQL 15 + RLS	Persistencia y seguridad
Realtime	Supabase Realtime (WebSockets)	Actualización en vivo
Storage	Supabase Storage	Fotografías (bucket público)
Auth	Supabase Auth (email/contraseña)	Autenticación y sesiones
Imágenes	sharp (backend) + browser-image-compression (front)	Compresión
Scripts	Python + Pillow + pillow-heif + supabase-py	Carga masiva
Hosting	GitHub Pages (front) + Railway (back)	Despliegue
CI/CD	GitHub Actions	Despliegue automático

5. Arquitectura del sistema

5.1 Arquitectura de alto nivel

El frontend Svelte (alojado en GitHub Pages) se comunica directamente con Supabase para Realtime, Storage y Auth, y con el backend Node.js/Fastify para la lógica de negocio compleja (productos, ventas, reportes e imágenes). El backend accede a Supabase mediante la clave de servicio (service/secret key).

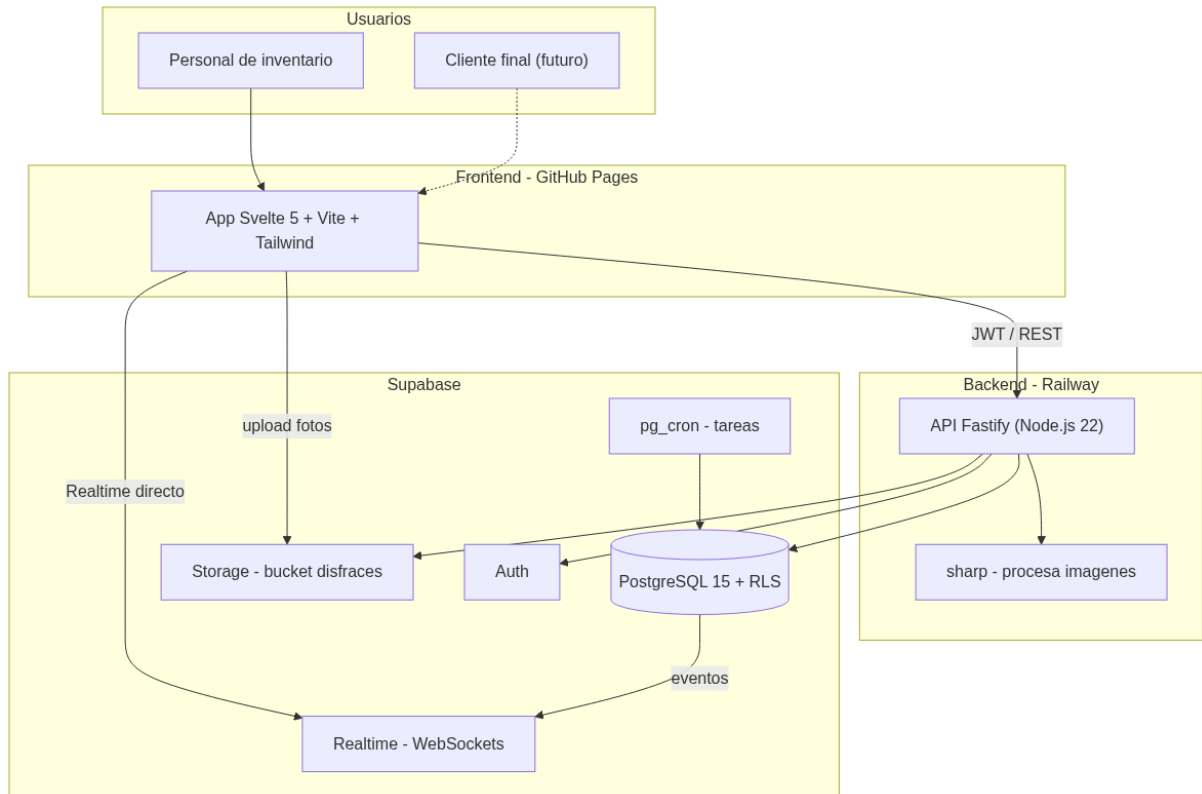


Diagrama 5.1 — Arquitectura general del sistema

5.2 Diagrama de componentes (bajo nivel)

Detalla los módulos internos del frontend (vistas y listener Realtime), del backend (módulos por recurso y procesamiento de imágenes) y de Supabase (base de datos, storage, realtime y auth).

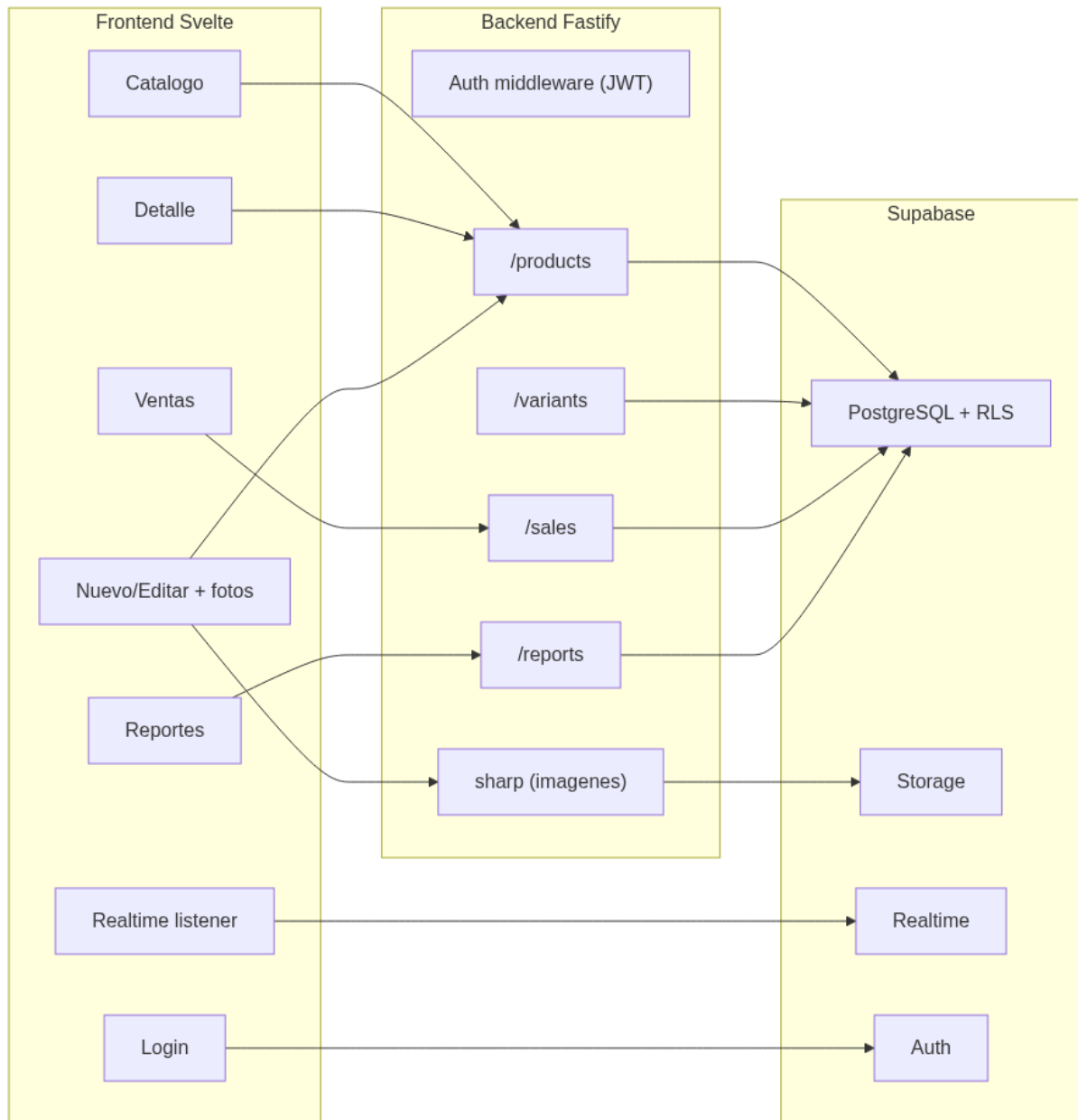


Diagrama 5.2 — Componentes internos y sus conexiones

6. Diagramas de flujo

6.1 Flujo de subida de fotografías

El usuario selecciona o toma una foto; el frontend la comprime a WebP con browser-image-compression y la envía al backend, que con sharp la redimensiona y la almacena en Supabase Storage, registrando la URL en la tabla product_images.

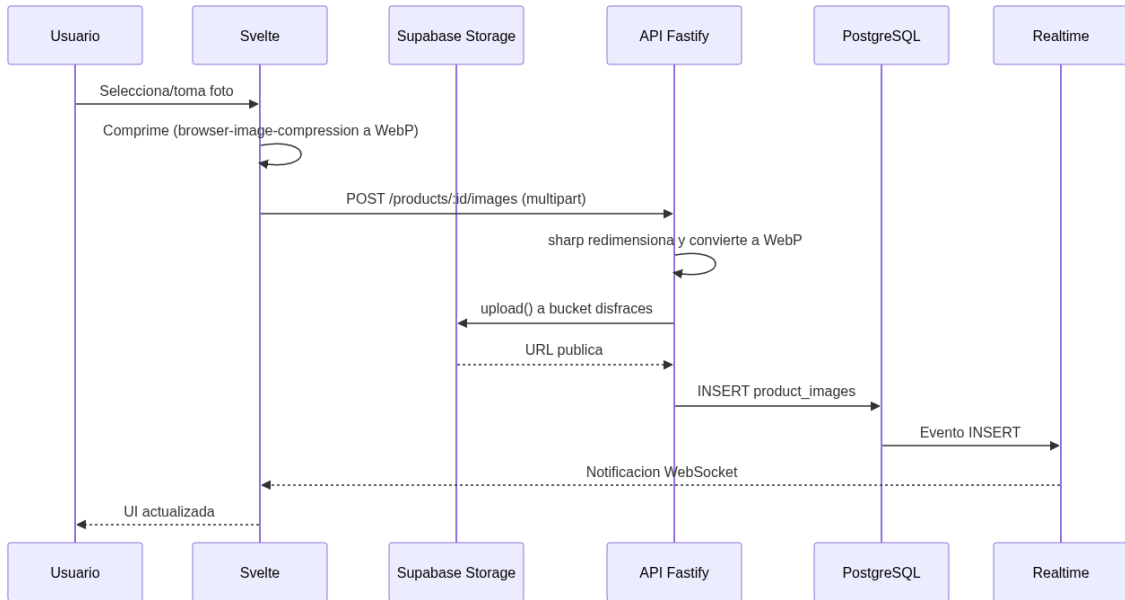


Diagrama 6.1 — Flujo de subida de fotografías

6.2 Flujo de stock en tiempo real

Al registrar una venta, el backend inserta la venta y sus ítems; un trigger de PostgreSQL valida y descuenta el stock en variants, recalcula el total y emite un evento Realtime que actualiza a todos los clientes conectados sin recargar.

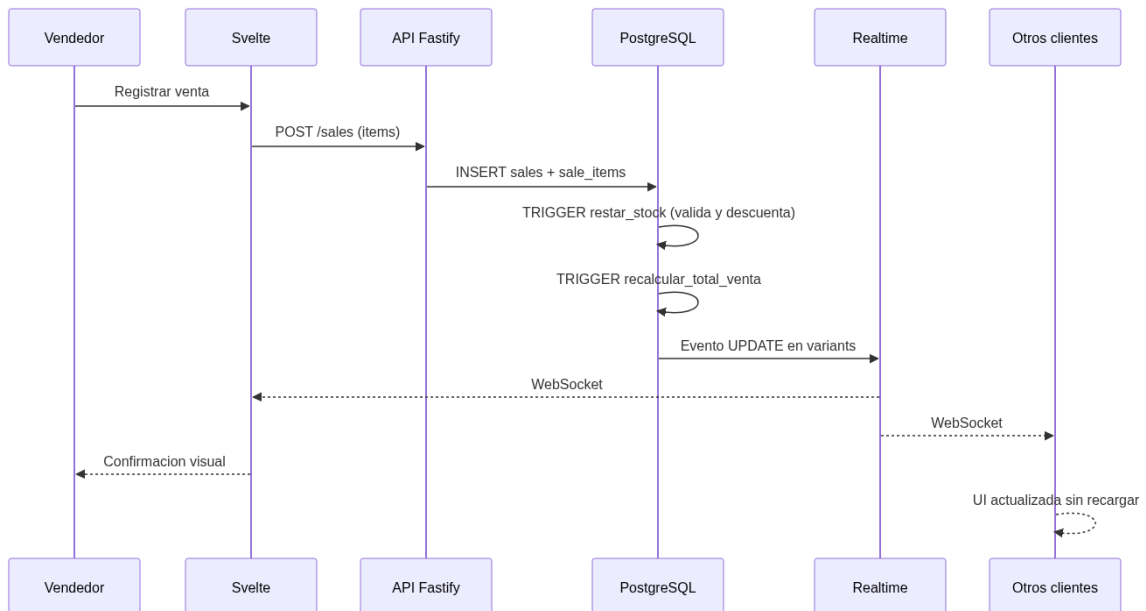


Diagrama 6.2 — Flujo de actualización de stock

7. Criterios de trazabilidad y lógica de negocio

7.1 Identificadores

Cada entidad utiliza un UUID como clave primaria (`gen_random_uuid()`), lo que garantiza unicidad global y trazabilidad de cada registro entre frontend, backend y base de datos. Las variantes incorporan además un SKU legible y único.

7.2 Generación automática de SKU

El SKU se genera en el backend a partir de variables que se mantienen constantes: el prefijo de la categoría (3 letras), el género y la talla. Si ya existe, se añade un sufijo numérico para garantizar la unicidad.

```
// backend: variantService.js
const GENERO_SKU = { 'niño': 'NIN', 'niña': 'NINA', 'hombre': 'HOM', 'mujer': 'MUJ', 'unisex': 'UNI' };

async function generarSku(productId, size, gender) {
  const prefijo = codigo3(categoria); // p.ej. "PEL" (películas)
  const generoCod = GENERO_SKU[gender] ?? 'GEN';
  const tallaCod = (slugify(size) || 'u').toUpperCase();
  let base = `${prefijo}-${generoCod}-${tallaCod}`; // p.ej. PEL-UNI-S
  let sku = base, n = 2;
  while (existeSku(sku)) sku = `${base}-${n++}`; // unicidad
  return sku;
}
```

Ejemplos reales: PEL-UNI-S, SUP-NIN-4, HOM-M. El SKU es la clave de trazabilidad operativa (ventas, inventario físico, reportes).

7.3 Lógica condicional (switch / categorización)

La clasificación de categorías y los códigos de género se resuelven con lógica de tipo switch/case (mapas de equivalencia), lo que permite estandarizar y trazar los valores.

```
// Carga masiva (Python): normalizacion de categoria (switch/case logico)
ALIAS_CATEGORIA = { "halloween": "halloween" }
def normalizar_slug_categoria(valor):
    slug = slugify(valor)
    return ALIAS_CATEGORIA.get(slug, slug)

# Frontend (Svelte): genero -> etiqueta
const GENEROS = [
  { valor: 'hombre', texto: 'Hombre' },
  { valor: 'mujer', texto: 'Mujer' },
  { valor: 'unisex', texto: 'Unisex' },
]

// Backend: filtros del catalogo (relacion categorias!inner)
if (category) select = select.replace('categories', 'categories!inner');
if (q) query = query.ilike('name', `%%${q}%`);
if (gender) items = items.filter(p => p.variants.some(v => v.gender === gender));
```

7.4 Triggers de base de datos (trazabilidad automática)

Cinco funciones PL/pgSQL garantizan la integridad y trazabilidad del inventario:

Trigger	Evento	Propósito
restar_stock	AFTER INSERT en sale_items	Valida stock (con FOR UPDATE) y lo descuenta.
devolver_stock	AFTER DELETE en sale_items	Devuelve el stock al eliminar un ítem.
recalcular_total_venta	AFTER I/U/D en sale_items	Recalcula el monto total de la venta.
handle_new_user	AFTER INSERT en auth.users	Crea el perfil del usuario automáticamente.
set_updated_at	BEFORE UPDATE	Mantiene la marca de actualización (updated_at).

```
create or replace function public.restar_stock()
returns trigger language plpgsql security definer set search_path = public as $$
declare v_stock integer;
begin
  select stock_quantity into v_stock
  from public.variants where id = new.variant_id for update;
  if not found then raise exception 'La variante % no existe', new.variant_id; end if;
  if v_stock < new.quantity then
    raise exception 'Stock insuficiente: disponible %, solicitado %', v_stock, new.quantity;
  end if;
  update public.variants
  set stock_quantity = stock_quantity - new.quantity, updated_at = now()
  where id = new.variant_id;
  return new;
end; $$;
```

7.5 Matriz de trazabilidad

Requisito / caso de uso	Componente	Dato trazado	Resultado esperado
Registrar venta y descontar stock	POST /sales + trigger restar_stock	variants.stock_quantity	Venta reflejada y stock reducido en vivo
Subir foto con compresión	POST /products/:id/images (multipart)	product_images.image_url	Foto almacenada y visible
Buscar/filtrar catálogo	GET /products?q&category&gender	products, variants, categories	Listado filtrado
Bajo stock	GET /reports/low-stock	variants.stock_quantity	Variantes $\leq$ umbral
Resumen de ventas	GET /reports/sales-summary	sales.total_amount	Totales por periodo
Crear/editar producto	POST/PUT /products	products	Producto guardado

SKU automático	POST /variants (generarSku)	variants.sku	SKU único generado
---------------------------	------------------------------------	---------------------	-------------------------------

8. Modelo de datos

8.1 Diagrama Entidad-Relación

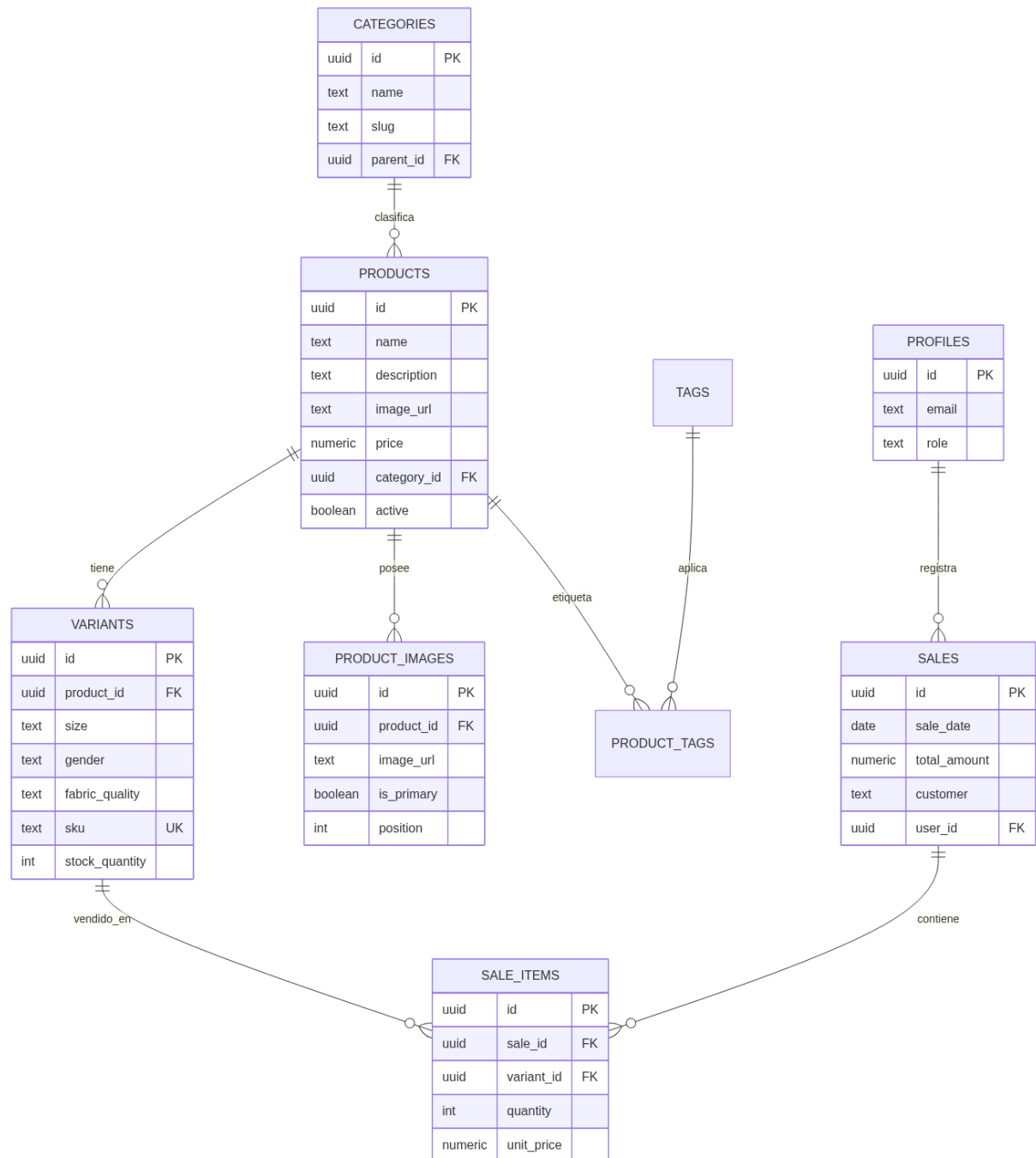


Diagrama 8.1 — Modelo Entidad-Relación

8.2 Diccionario de tablas

Tabla	Campos principales	Descripción
categories	id (UUID, PK), name, slug (UK), parent_id (FK), created_at	Jerarquía de temáticas
products	id (UUID, PK), name, description, image_url, price, category_id (FK), active, timestamps	Cada disfraz
variants	id (UUID, PK), product_id (FK), size, gender, fabric_quality, sku (UK), stock_quantity	Talla + género + tela + stock
product_images	id (UUID, PK), product_id (FK), image_url, is_primary, position	Fotografías
tags / product_tags	id (UUID, PK) / (product_id, tag_id)	Etiquetas flexibles
sales	id (UUID, PK), sale_date, total_amount, customer, user_id (FK)	Ventas
sale_items	id (UUID, PK), sale_id (FK), variant_id (FK), quantity, unit_price	Detalle de venta
profiles	id (UUID, PK = auth.users), email, full_name, role	Usuarios del sistema

9. Backend (Node.js + Fastify)

9.1 Estructura del proyecto

inventario-backend/	
├─ src/server.js	Servidor Fastify, CORS, helmet, multipart, errores
├─ src/config/env.js	Variables de entorno validadas
├─ src/lib/supabase.js	Cliente admin (service/secret key)
├─ src/middlewares/auth.js	requireAuth / requireAdmin (JWT de Supabase)
├─ src/schemas/*.js	Validacion con zod
├─ src/services/*.js	Logica de negocio (product/variant/sale/report)
├─ src/routes/*.js	Endpoints REST

9.2 Endpoints

Método	Ruta	Auth	Descripción
GET	/health	No	Estado del servicio
GET	/products	No	Listado con filtros (q,

			category, gender, page)
GET	/products/:id	No	Detalle con imágenes y variantes
POST/PUT/DELETE	/products(/:id)	Sí	Alta, edición y baja de productos
POST	/products/:id/images	Sí	Subir imagen (sharp) a Storage
GET	/variants	No	Variantes (filtros product_id, gender, low_stock)
POST/PATCH	/variants(/:id)	Sí	Crear/ajustar variante (SKU automático)
GET/POST	/sales	Sí	Listar/registrar ventas
GET	/reports/low-stock	Sí	Reporte de bajo stock
GET	/reports/sales-summary	Sí	Resumen de ventas
GET	/reports/top-products	Sí	Productos más vendidos

9.3 Funciones principales

Autenticación (JWT de Supabase) y control por rol:

```
export async function requireAuth(request) {
  const token = (request.headers.authorization ?? '').replace('Bearer ', '');
  const { data, error } = await supabaseAdmin.auth.getUser(token);
  if (error || !data?.user) throw new AppError('Token invalido o expirado', 401);
  request.user = { id: data.user.id, email: data.user.email, role: perfil?.role };
}
```

Registro de venta (transaccional, con deshacer ante error):

```
export async function createSale(userId, input) {
  const { data: venta } = await supabaseAdmin.from('sales')
    .insert({ customer, sale_date, total_amount: 0, user_id: userId })
    .select('id').single();

  const filas = items.map(i => ({ sale_id: venta.id, variant_id: i.variant_id,
    quantity: i.quantity, unit_price: i.unit_price }));
  const { error } = await supabaseAdmin.from('sale_items').insert(filas);
  if (error) {
    // p.ej. stock insuficiente (trigger)
    await supabaseAdmin.from('sales').delete().eq('id', venta.id); // rollback
    throw new AppError(`No se pudo registrar la venta: ${error.message}`, 409);
  }
  return getSale(venta.id); // triggers ya restaron stock y calcularon total
}
```

Subida de imagen con sharp (redimensiona y convierte a WebP):

```
export async function subirImagen(productId, buffer, { isPrimary } = {}) {
  const webp = await sharp(buffer).rotate()
    .resize({ width: 1600, height: 1600, fit: 'inside', withoutEnlargement: true })
    .webp({ quality: 80 }).toBuffer();
  const ruta = `productos/${slug}/${Date.now()}-${slug}.webp`;
  await supabaseAdmin.storage.from(bucket).upload(ruta, webp, { contentType: 'image/webp',
    upsert: true });
  const url = supabaseAdmin.storage.from(bucket).getPublicUrl(ruta).data.publicUrl;
  // La primera imagen del producto se marca como principal
  await supabaseAdmin.from('product_images').insert({ product_id, image_url: url, is_primary:
    principal, position: count });
  return img;
}
```

10. Frontend (Svelte 5 + Vite + Tailwind)

10.1 Estructura

```
inventario-frontend/src/
├── App.svelte                Shell + router por hash + guard de sesion
├── lib/supabase.js           Cliente Supabase (anon key)
├── lib/api.js                Cliente del backend (adjunta el JWT)
├── lib/auth.svelte.js        Estado de sesión (runes)
├── lib/router.svelte.js      Router por hash (compatible GitHub Pages)
├── lib/components/Nav.svelte Barra de navegación
├── lib/components/ImagePicker.svelte Cámara/galería + compresión + preview
├── lib/components/Lightbox.svelte Zoom de imágenes
├── routes/*.svelte           Login, Catalogo, Detalle, Editar, Ventas, Reportes
```

10.2 Rutas y vistas

Ruta	Vista	Descripción
#/	Catálogo	Lista con filtros y tarjetas de producto
#/producto/:id	Detalle	Ficha, variantes, stock y zoom de fotos
#/producto/nuevo	Alta	Formulario + variante + tela + fotos
#/producto/:id/editar	Edición	Editar datos y agregar fotos
#/ventas	Ventas	Carrito y registro de venta
#/reportes	Reportes	Bajo stock, resumen y más vendidos

10.3 Componentes clave

ImagePicker: Botones Tomar foto / Galería, comprime a WebP (0.5 MB, 1600px), previsualiza y limita de 1 a 3 fotos.

Lightbox: Visor a pantalla completa con zoom (100%–400%) para observar la imagen sin recorte.

api.js: Centraliza las llamadas al backend añadiendo el token JWT de Supabase.

auth.svelte.js: Maneja la sesión con Supabase Auth y expone estado reactivo.

router.svelte.js: Router por hash: funciona en GitHub Pages sin configuración de servidor.

11. Mockups

11.1 Baja fidelidad (wireframes)

Bocetos de estructura y distribución de los elementos en cada pantalla, sin color ni estilos, para validar la organización de la información.

A wireframe of a login form. At the top is a dark gray square placeholder for a logo. Below it is the title 'Inventario de Disfraces' in bold, followed by the subtitle 'Control interno' in a smaller font. The form contains two input fields: the first is labeled 'Correo' and contains the text 'tu@correo.com'; the second is labeled 'Contraseña' and contains a series of dots representing a password. Below these fields is a dark gray button with the text 'Ingresar'.

Wireframe — Login

Catálogo de disfraces

+ Nuevo producto

Buscar

por nombre...

Todas las categorías v

Todos los géneros v

Buscar

Limpiar

Capitán América

S/ 45.00

8 und.

Spiderman

S/ 40.00

8 und.

Mujer Maravilla

S/ 40.00

8 und.

Gekko

S/ 50.00

8 und.

Maléfica

S/ 150.00

8 und.

Skye Paw Patrol

S/ 45.00

8 und.

Blanca Nieves

S/ 45.00

8 und.

Gatúbela

S/ 90.00

8 und.

Wireframe — Catálogo

← Volver

Ampliar

Capitán América

Superhéroes

S/ 45.00

Tallas / variantes

SKU	Talla	Género	Tela	Stock
SUP-NIN-4	4	niño	polinan	5
SUP-NIN-6	6	niño	polinan	4
SUP-NIN-8	8	niño	polinan	3

Editar

Eliminar

Wireframe — Detalle de producto

Nuevo producto

Nombre

Capitán América

Precio (\$)

45.00

Categoría

Superhéroes v

Variante (talla / género / stock)

Talla

4 v

Género

Niño v

Cantidad

5

☐ Comentar calidad de la tela (opcional)

☐ Agregar comentario (opcional)

☒ Activo

Fotos (1 a 3)

Tomar foto

Desde galería

Crear y subir fotos

Wireframe — Alta de producto

Inventario de Disfraces

Catálogo

Ventas

Reportes

salir

Registrar venta

Agregar producto

Spiderman - 6 - niño (stock 6) v

Cantidad

1

Agregar

Últimas ventas

04/10/2026	Cliente 1	\$/ 40.00
05/10/2026	Cliente 2	\$/ 50.00
06/10/2026	Cliente 3	\$/ 60.00

Carrito (1)

Spiderman - 6 - niño

1 × \$/ 40.00

Cliente (opcional)

Total: \$/ 40.00

Registrar venta

Wireframe — Ventas

Reportes

Desde
2026-10-01

Hasta
2026-10-01

Aplicar

Ventas en el periodo
18

Monto total
S/ 1,240.00

Bajo stock (<= 3)

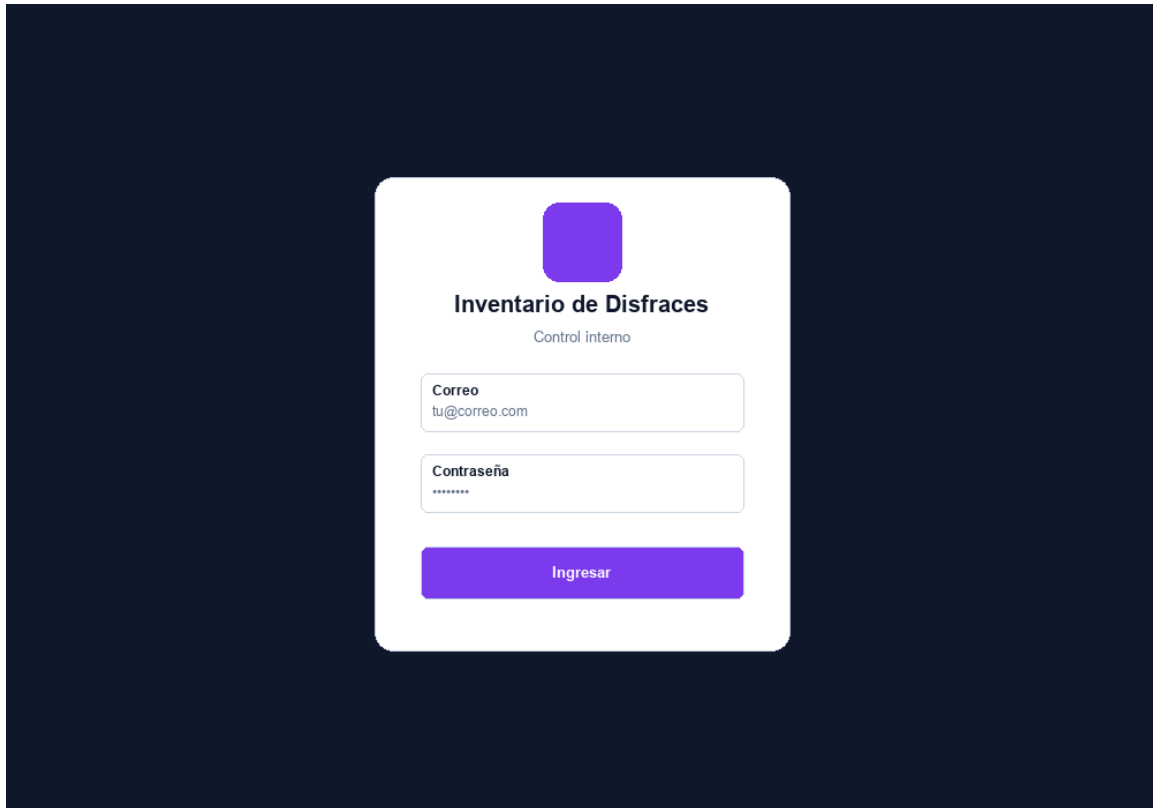
Gatúbela · Small	1
Reo americano · M	2
Gekko · 8	3
Lady boo · 4	3

Más vendidos

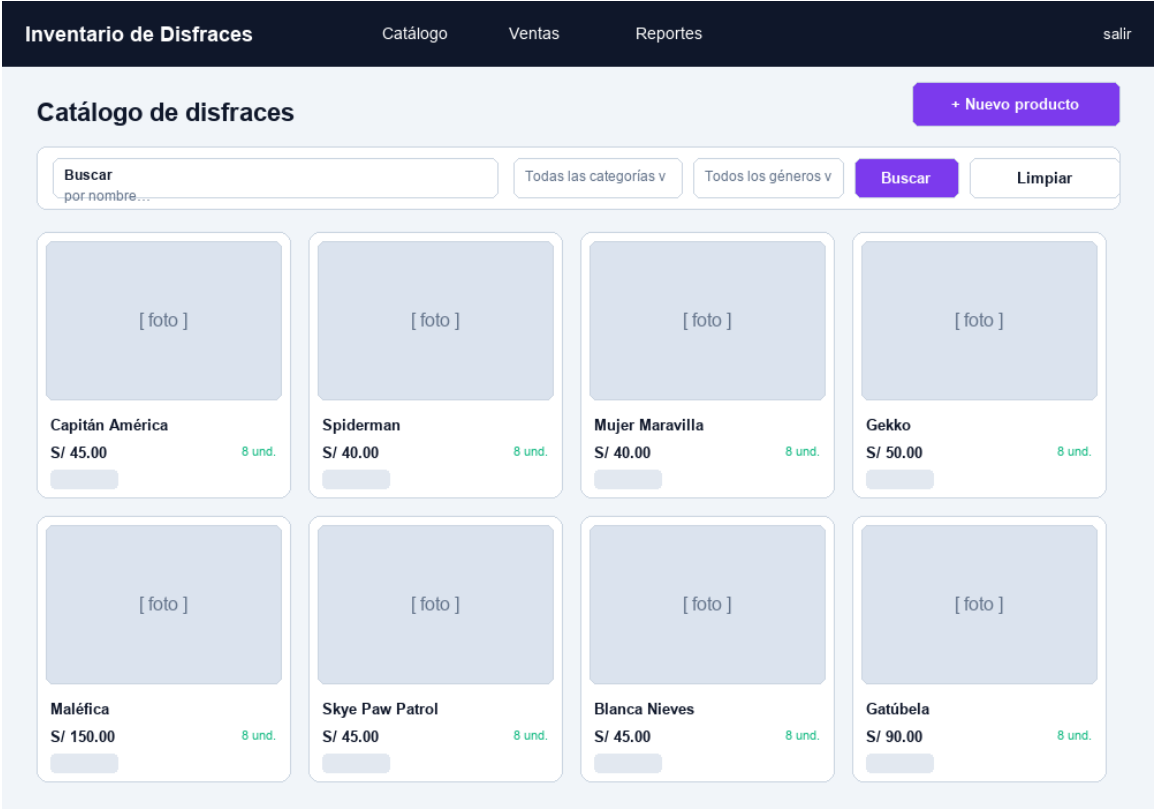
Spiderman sublimado	12 und.
Capitán América	9 und.
Maléfica	7 und.
Skye Paw Patrol	5 und.

11.2 Alta fidelidad

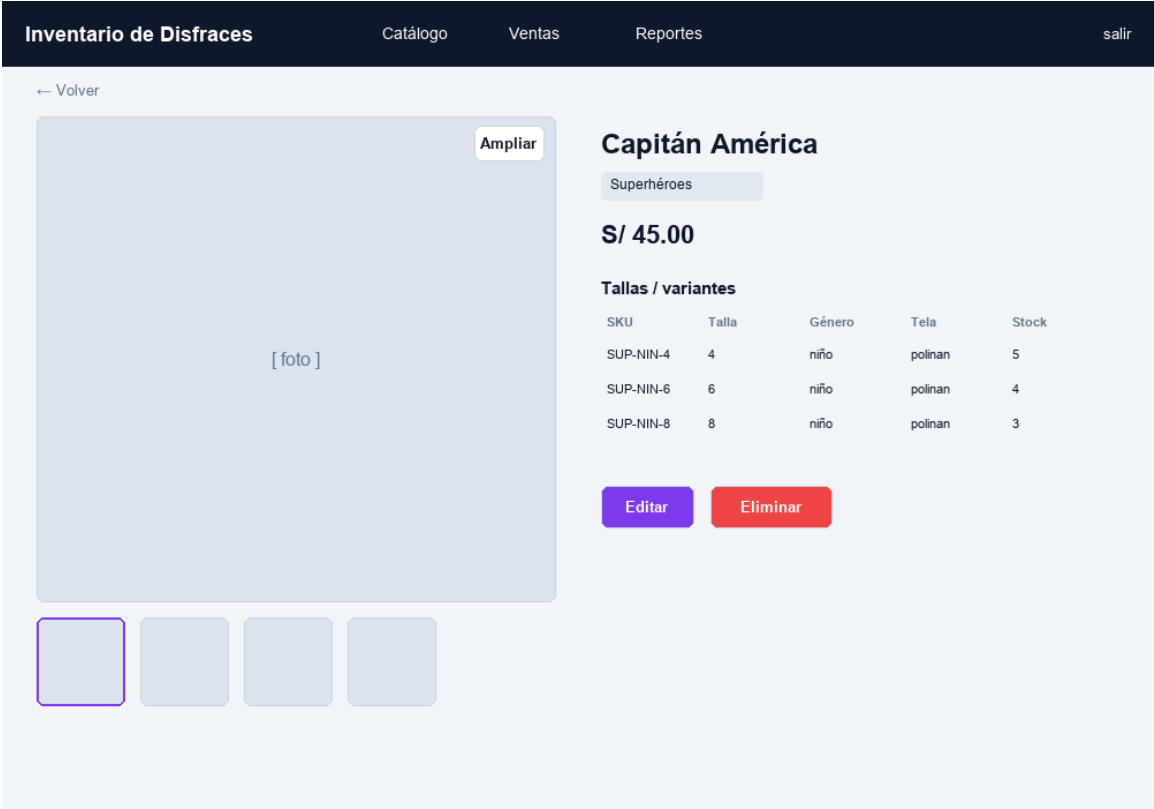
Diseño final con la paleta y componentes reales (Svelte + Tailwind), cercano a la apariencia de la aplicación.



Mockup — Login



Mockup — Catálogo



Inventario de Disfraces

Catálogo

Ventas

Reportes

salir

Nuevo producto

Nombre
Capitán América

Precio (\$)
45.00

Categoría
Superhéroes v

Variante (talla / género / stock)

Talla
4 v

Género
Niño v

Cantidad
5

☐ Comentar calidad de la tela (opcional)

☐ Agregar comentario (opcional)

☒ Activo

Fotos (1 a 3)

Tomar foto

Desde galería

[foto]

[foto]

[foto]

Crear y subir fotos

Inventario de Disfraces

Catálogo

Ventas

Reportes

salir

Registrar venta

Agregar producto

Spiderman · 6 · niño (stock 6) v

Cantidad

1

Agregar

Últimas ventas

04/10/2026	Cliente 1	S/ 40.00
05/10/2026	Cliente 2	S/ 50.00
06/10/2026	Cliente 3	S/ 60.00

Carrito (1)

Spiderman · 6 · niño

1 × S/ 40.00

Cliente (opcional)

Total: S/ 40.00

Registrar venta

Mockup — Ventas

Inventario de Disfraces

Catálogo

Ventas

Reportes

salir

Reportes

Desde

2026-10-01

Hasta

2026-10-01

Aplicar

Ventas en el periodo

18

Monto total

S/ 1,240.00

Bajo stock (<= 3)

Gatúbela · Small	1
Reo americano · M	2
Gekko · 8	3
Lady boo · 4	3

Más vendidos

Spiderman sublimado	12 und.
Capitán América	9 und.
Maléfica	7 und.
Skye Paw Patrol	5 und.

12. Seguridad

- Row Level Security (RLS) en todas las tablas: lectura pública del catálogo y escritura solo para usuarios autenticados.
- Autenticación con Supabase Auth (email/contraseña) y verificación del token JWT en el backend.
- La clave de servicio (service/secret key) se usa solo en el backend y nunca se expone en el frontend.
- Variables sensibles almacenadas en archivos .env excluidos del repositorio (.gitignore).
- Bucket de Storage público solo para lectura; escritura restringida a usuarios autenticados.
- Validación de datos con zod en todos los endpoints.

13. Despliegue

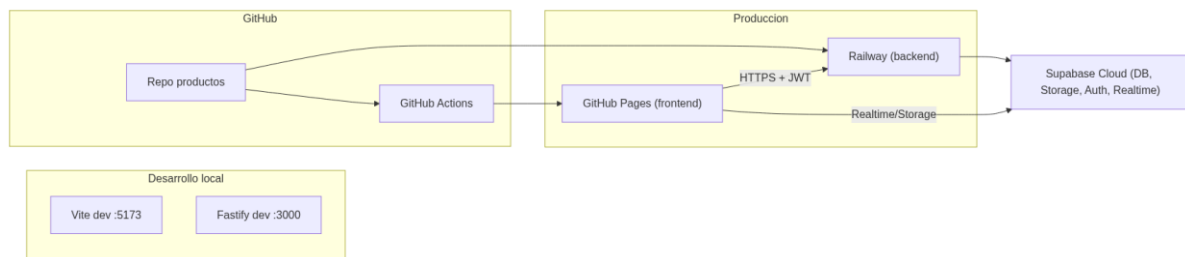


Diagrama 13.1 — Despliegue en la nube

Componente	Plataforma	URL / detalle	Notas
Frontend	GitHub Pages	https://disfracesrosaelvira.github.io/productos /	Deploy automático con GitHub Actions
Backend	Railway	https://productos-production-8443.up.railway.app	Node.js 22, variables de entorno
Base de datos	Supabase (proyecto InventorySystem)	https://zlcjzgpcswigxulspjpp.supabase.co	PostgreSQL, Storage, Auth, Realtime
CI/CD	GitHub Actions	deploy-frontend.yml	Build y publicación del frontend

14. Manual de uso

- 1. Iniciar sesión con el correo y contraseña asignados.
- 2. En el Catálogo, buscar por nombre o filtrar por categoría/género. Usar 'Limpiar filtros' para reiniciar.
- 3. Pulsar '+ Nuevo producto' para dar de alta: completar nombre, precio, categoría y categoría; elegir talla, género, cantidad y (opcional) la calidad de la tela.
- 4. Añadir de 1 a 3 fotos con 'Tomar foto' (cámara) o 'Desde galería'. Se comprimen automáticamente.
- 5. Guardar; el sistema crea el producto, su variante con SKU automático y sube las fotos.
- 6. En el Detalle, usar '🔍 Ampliar' para ver la imagen completa con zoom.
- 7. En Ventas, seleccionar la variante, indicar cantidad, agregarla al carrito y registrar la venta (el stock se descuenta solo).
- 8. En Reportes, consultar bajo stock, resumen de ventas y productos más vendidos.

15. Anexos

Anexo A. Guion SQL de la base de datos

Script completo ejecutado en Supabase (db/supabase_schema.sql):

```
-- =====
-- Proyecto: Sistema de Inventario de Disfraces
-- Archivo : supabase_schema.sql
-- Fase   : 1 - Base de Datos (Semana 1)
-- Uso    : Supabase Dashboard -> SQL Editor -> New query -> pegar -> Run
-- =====

-- =====
-- 0. EXTENSIONES
-- =====
create extension if not exists pgcrypto; -- gen_random_uuid()

-- NOTA: pg_cron NO se crea aqui. Se habilita en Supabase desde:
-- Dashboard -> Database -> Extensions -> buscar "pg_cron" -> Enable
-- Solo es necesario para las tareas programadas de la Fase 6.
-- create extension if not exists pg_cron;

-- =====
-- 1. TABLAS
-- =====

-- 1.1 PROFILES (perfil de usuario del sistema)
create table if not exists public.profiles (
  id          uuid primary key references auth.users (id) on delete cascade,
  email       text,
  full_name   text,
  role        text not null default 'vendedor'
              check (role in ('admin', 'vendedor', 'consulta')),
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now()
);

-- 1.2 CATEGORIES (jerarquia de tematicas)
create table if not exists public.categories (
```

```

    id          uuid primary key default gen_random_uuid(),
    name        text not null,
    slug        text unique,
    parent_id   uuid references public.categories (id) on delete set null,
    created_at  timestampz not null default now()
);

-- 1.3 PRODUCTS (cada disfraz)
create table if not exists public.products (
    id          uuid primary key default gen_random_uuid(),
    name        text not null,
    description text,
    image_url   text,                -- foto principal (URL publica de Storage)
    price       numeric(12, 2),      -- precio de referencia en soles
    category_id uuid references public.categories (id) on delete set null,
    active      boolean not null default true,
    created_at  timestampz not null default now(),
    updated_at  timestampz not null default now()
);

-- Salvaguarda por si la tabla ya existia sin la columna price
alter table public.products add column if not exists price numeric(12, 2);

-- 1.4 VARIANTS (talla + genero + tela + stock)
create table if not exists public.variants (
    id          uuid primary key default gen_random_uuid(),
    product_id  uuid not null references public.products (id) on delete cascade,
    size        text,                -- S, M, L, XL, 4, 6, 8...
    gender      text
        check (gender in ('niño', 'niña', 'hombre', 'mujer', 'unisex')),
    fabric_quality text,             -- economica, estandar, premium
    sku         text unique,
    stock_quantity integer not null default 0 check (stock_quantity >= 0),
    created_at  timestampz not null default now(),
    updated_at  timestampz not null default now()
);

-- 1.5 PRODUCT_IMAGES (fotos adicionales)
create table if not exists public.product_images (
    id          uuid primary key default gen_random_uuid(),
    product_id  uuid not null references public.products (id) on delete cascade,
    image_url   text not null,
    is_primary  boolean not null default false,
    position    integer not null default 0,
    created_at  timestampz not null default now()
);

-- 1.6 TAGS (etiquetas flexibles)
create table if not exists public.tags (
    id          uuid primary key default gen_random_uuid(),
    name        text not null unique,
    created_at  timestampz not null default now()
);

-- 1.7 PRODUCT_TAGS (N:M producto <-> tag)
create table if not exists public.product_tags (
    product_id  uuid not null references public.products (id) on delete cascade,
    tag_id      uuid not null references public.tags (id) on delete cascade,
    primary key (product_id, tag_id)
);

-- 1.8 SALES (ventas)
create table if not exists public.sales (
    id          uuid primary key default gen_random_uuid(),
    sale_date   date not null default current_date,
    total_amount numeric(12, 2) not null default 0,
    customer    text,
    user_id     uuid references public.profiles (id) on delete set null,
    created_at  timestampz not null default now()
);

-- 1.9 SALE_ITEMS (detalle de venta)
create table if not exists public.sale_items (

```

```

        id          uuid primary key default gen_random_uuid(),
        sale_id     uuid not null references public.sales (id) on delete cascade,
        variant_id  uuid not null references public.variants (id) on delete restrict,
        quantity    integer not null check (quantity > 0),
        unit_price  numeric(12, 2) not null default 0,
        created_at  timestamptz not null default now()
    );

-- =====
-- 2. INDICES
-- =====
create index if not exists idx_products_category    on public.products (category_id);
create index if not exists idx_variants_product    on public.variants (product_id);
create index if not exists idx_variants_size       on public.variants (size);
create index if not exists idx_variants_gender     on public.variants (gender);
create index if not exists idx_images_product      on public.product_images (product_id);
create index if not exists idx_sales_user          on public.sales (user_id);
create index if not exists idx_sales_date          on public.sales (sale_date);
create index if not exists idx_sale_items_sale     on public.sale_items (sale_id);
create index if not exists idx_sale_items_variant  on public.sale_items (variant_id);

-- =====
-- 3. FUNCIONES Y TRIGGERS
-- =====

-- 3.1 updated_at automatico
create or replace function public.set_updated_at()
returns trigger
language plpgsql
as $$
begin
    new.updated_at = now();
    return new;
end;
$$;

drop trigger if exists trg_profiles_updated on public.profiles;
create trigger trg_profiles_updated
before update on public.profiles
for each row execute function public.set_updated_at();

drop trigger if exists trg_products_updated on public.products;
create trigger trg_products_updated
before update on public.products
for each row execute function public.set_updated_at();

drop trigger if exists trg_variants_updated on public.variants;
create trigger trg_variants_updated
before update on public.variants
for each row execute function public.set_updated_at();

-- 3.2 Crear perfil automaticamente al registrar usuario en Auth
create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
    insert into public.profiles (id, email, full_name, role)
    values (
        new.id,
        new.email,
        coalesce(new.raw_user_meta_data ->> 'full_name', ''),
        'vendedor'
    )
    on conflict (id) do nothing;
    return new;
end;
$$;

```

```

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();

-- 3.3 Restar stock al insertar en sale_items
-- Usa FOR UPDATE para evitar condiciones de carrera y valida stock.
create or replace function public.restar_stock()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
declare
  v_stock integer;
begin
  select stock_quantity
    into v_stock
    from public.variants
   where id = new.variant_id
   for update;

  if not found then
    raise exception 'La variante % no existe', new.variant_id;
  end if;

  if v_stock < new.quantity then
    raise exception 'Stock insuficiente para la variante %: disponible %, solicitado %',
      new.variant_id, v_stock, new.quantity;
  end if;

  update public.variants
    set stock_quantity = stock_quantity - new.quantity,
        updated_at      = now()
    where id = new.variant_id;

  return new;
end;
$$;

drop trigger if exists trg_restar_stock on public.sale_items;
create trigger trg_restar_stock
  after insert on public.sale_items
  for each row execute function public.restar_stock();

-- 3.4 Devolver stock al eliminar un sale_item (revierte la operacion)
create or replace function public.devolver_stock()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  update public.variants
    set stock_quantity = stock_quantity + old.quantity,
        updated_at      = now()
    where id = old.variant_id;

  return old;
end;
$$;

drop trigger if exists trg_devolver_stock on public.sale_items;
create trigger trg_devolver_stock
  after delete on public.sale_items
  for each row execute function public.devolver_stock();

-- 3.5 Recalcular total de la venta al cambiar sus items
create or replace function public.recalcular_total_venta()
returns trigger

```

```

language plpgsql
security definer
set search_path = public
as $$
declare
    v_sale_id uuid := coalesce(new.sale_id, old.sale_id);
begin
    update public.sales
        set total_amount = coalesce(
            (select sum(quantity * unit_price)
             from public.sale_items
             where sale_id = v_sale_id), 0)
        where id = v_sale_id;

    return null;
end;
$$;

drop trigger if exists trg_recalcular_total on public.sale_items;
create trigger trg_recalcular_total
    after insert or update or delete on public.sale_items
    for each row execute function public.recalcular_total_venta();

-- 3.6 Helper de rol admin (SECURITY DEFINER evita recursion en RLS)
create or replace function public.is_admin()
returns boolean
language sql
security definer
stable
set search_path = public
as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role = 'admin'
    );
$$;

-- =====
-- 4. ROW LEVEL SECURITY (RLS)
--   Lectura publica, escritura autenticada.
-- =====
alter table public.profiles      enable row level security;
alter table public.categories    enable row level security;
alter table public.products      enable row level security;
alter table public.variants      enable row level security;
alter table public.product_images enable row level security;
alter table public.tags          enable row level security;
alter table public.product_tags  enable row level security;
alter table public.sales         enable row level security;
alter table public.sale_items    enable row level security;

-- 4.1 Catalogos: lectura publica (anon + authenticated)
drop policy if exists "lectura publica categories" on public.categories;
create policy "lectura publica categories"
    on public.categories for select to anon, authenticated using (true);

drop policy if exists "lectura publica products" on public.products;
create policy "lectura publica products"
    on public.products for select to anon, authenticated using (true);

drop policy if exists "lectura publica variants" on public.variants;
create policy "lectura publica variants"
    on public.variants for select to anon, authenticated using (true);

drop policy if exists "lectura publica product_images" on public.product_images;
create policy "lectura publica product_images"
    on public.product_images for select to anon, authenticated using (true);

drop policy if exists "lectura publica tags" on public.tags;

```

```

create policy "lectura publica tags"
  on public.tags for select to anon, authenticated using (true);

drop policy if exists "lectura publica product_tags" on public.product_tags;
create policy "lectura publica product_tags"
  on public.product_tags for select to anon, authenticated using (true);

-- 4.2 Catalogos: escritura solo autenticados
drop policy if exists "escritura auth categories" on public.categories;
create policy "escritura auth categories"
  on public.categories for all to authenticated using (true) with check (true);

drop policy if exists "escritura auth products" on public.products;
create policy "escritura auth products"
  on public.products for all to authenticated using (true) with check (true);

drop policy if exists "escritura auth variants" on public.variants;
create policy "escritura auth variants"
  on public.variants for all to authenticated using (true) with check (true);

drop policy if exists "escritura auth product_images" on public.product_images;
create policy "escritura auth product_images"
  on public.product_images for all to authenticated using (true) with check (true);

drop policy if exists "escritura auth tags" on public.tags;
create policy "escritura auth tags"
  on public.tags for all to authenticated using (true) with check (true);

drop policy if exists "escritura auth product_tags" on public.product_tags;
create policy "escritura auth product_tags"
  on public.product_tags for all to authenticated using (true) with check (true);

-- 4.3 Profiles: cada usuario ve y edita su propio perfil; los admin ven todo
drop policy if exists "perfil propio select" on public.profiles;
create policy "perfil propio select"
  on public.profiles for select to authenticated
  using (id = auth.uid() or public.is_admin());

drop policy if exists "perfil propio update" on public.profiles;
create policy "perfil propio update"
  on public.profiles for update to authenticated
  using (id = auth.uid())
  with check (id = auth.uid());

-- 4.4 Ventas: solo usuarios autenticados
drop policy if exists "ventas auth select" on public.sales;
create policy "ventas auth select"
  on public.sales for select to authenticated using (true);

drop policy if exists "ventas auth insert" on public.sales;
create policy "ventas auth insert"
  on public.sales for insert to authenticated with check (true);

drop policy if exists "ventas auth update" on public.sales;
create policy "ventas auth update"
  on public.sales for update to authenticated using (true) with check (true);

drop policy if exists "ventas auth delete" on public.sales;
create policy "ventas auth delete"
  on public.sales for delete to authenticated using (true);

drop policy if exists "items auth select" on public.sale_items;
create policy "items auth select"
  on public.sale_items for select to authenticated using (true);

drop policy if exists "items auth insert" on public.sale_items;
create policy "items auth insert"
  on public.sale_items for insert to authenticated with check (true);

```

```
drop policy if exists "items auth update" on public.sale_items;
create policy "items auth update"
    on public.sale_items for update to authenticated using (true) with check (true);
```

```
drop policy if exists "items auth delete" on public.sale_items;
create policy "items auth delete"
    on public.sale_items for delete to authenticated using (true);
```

```
-- =====
-- 5. REALTIME
-- =====
do $$
begin
    if not exists (select 1 from pg_publication_tables
                    where pubname = 'supabase_realtime'
                       and schemaname = 'public' and tablename = 'variants') then
        alter publication supabase_realtime add table public.variants;
    end if;
    if not exists (select 1 from pg_publication_tables
                    where pubname = 'supabase_realtime'
                       and schemaname = 'public' and tablename = 'products') then
        alter publication supabase_realtime add table public.products;
    end if;
    if not exists (select 1 from pg_publication_tables
                    where pubname = 'supabase_realtime'
                       and schemaname = 'public' and tablename = 'sales') then
        alter publication supabase_realtime add table public.sales;
    end if;
end $$;
```

```
-- Replicar el valor completo (UPDATE) en los eventos
alter table public.variants replica identity full;
alter table public.products replica identity full;
```

```
-- =====
-- 6. STORAGE (bucket publico de fotos)
-- =====
insert into storage.buckets (id, name, public)
values ('disfraces', 'disfraces', true)
on conflict (id) do update set public = true;
```

```
drop policy if exists "disfraces lectura publica" on storage.objects;
create policy "disfraces lectura publica"
    on storage.objects for select to anon, authenticated
    using (bucket_id = 'disfraces');
```

```
drop policy if exists "disfraces escritura auth" on storage.objects;
create policy "disfraces escritura auth"
    on storage.objects for insert to authenticated
    with check (bucket_id = 'disfraces');
```

```
drop policy if exists "disfraces update auth" on storage.objects;
create policy "disfraces update auth"
    on storage.objects for update to authenticated
    using (bucket_id = 'disfraces') with check (bucket_id = 'disfraces');
```

```
drop policy if exists "disfraces delete auth" on storage.objects;
create policy "disfraces delete auth"
    on storage.objects for delete to authenticated
    using (bucket_id = 'disfraces');
```

```
-- =====
-- 7. DATOS SEMILLA (categorias base del proyecto)
-- =====
insert into public.categories (name, slug) values
('Superheroes', 'superheroes'),
```

```

('Profesiones', 'profesiones'),
('Series', 'series'),
('Peliculas', 'peliculas'),
('Halloween', 'halloween')
on conflict (slug) do nothing;

-- =====
-- 8. VERIFICACION RAPIDA
-- =====
-- select table_name from information_schema.tables
--   where table_schema = 'public' order by table_name;
-- select id, name, public from storage.buckets;
-- select schemaname, tablename, rowsecurity from pg_tables
--   where schemaname = 'public' order by tablename;
-- =====
-- FIN
-- =====

```

Anexo B. Script de carga masiva (Python)

El script `scripts/cargar_supabase.py` comprime las fotografías a WebP, las sube al bucket, y crea/actualiza productos, imágenes y variantes (una por talla) leyendo el CSV de metadatos. Es idempotente (reejecutable).

```

# Fragmento clave: una variante por talla (stock por talla) y SKU unico
for talla in tallas_de(row):
    # "4;6;8" -> ["4","6","8"]
    sku = f"{prefijo}-{slugify(talla) or 'unica'}.upper()"
    client.table("variants").upsert({
        "product_id": prod_id, "size": talla, "gender": row["genero"],
        "fabric_quality": row.get("calidad_tela") or None,
        "sku": sku, "stock_quantity": stock,
    }, on_conflict="sku").execute()

```

Anexo C. Código Mermaid de los diagramas

Los diagramas pueden regenerarse pegando este código en mermaid.live:

Diagrama: arquitectura

```

graph TD
    subgraph TB
        subgraph Users["Usuarios"]
            U1["Personal de inventario"]
            U2["Cliente final (futuro)"]
        end
        subgraph FE["Frontend - GitHub Pages"]
            SV["App Svelte 5 + Vite + Tailwind"]
        end
        subgraph BE["Backend - Railway"]
            API["API Fastify (Node.js 22)"]
            SH["sharp - procesa imagenes"]
        end
        subgraph SB["Supabase"]
            PG["PostgreSQL 15 + RLS"]
            ST["Storage - bucket disfraces"]
            RT["Realtime - WebSockets"]
            AU["Auth"]
            CR["pg_cron - tareas"]
        end
        U1 --> SV
        U2 --> SV
        SV -- "JWT / REST" --> API
        SV -- "Realtime directo" --> RT
        SV -- "upload fotos" --> ST
        API --> PG
        API --> ST
    end

```

```
API --> AU
API --> SH
PG -- "eventos" --> RT
CR --> PG
```

Diagrama: subida_fotos

```
sequenceDiagram
    participant U as Usuario
    participant S as Svelte
    participant ST as Supabase Storage
    participant API as API Fastify
    participant DB as PostgreSQL
    participant RT as Realtime
    U->>S: Selecciona/toma foto
    S->>S: Comprime (browser-image-compression a WebP)
    S->>API: POST /products/:id/images (multipart)
    API->>API: sharp redimensiona y convierte a WebP
    API->>ST: upload() a bucket disfraces
    ST-->>API: URL publica
    API->>DB: INSERT product_images
    DB->>RT: Evento INSERT
    RT-->>S: Notificacion WebSocket
    S-->>U: UI actualizada
```

Diagrama: stock

```
sequenceDiagram
    participant V as Vendedor
    participant S as Svelte
    participant API as API Fastify
    participant DB as PostgreSQL
    participant RT as Realtime
    participant C as Otros clientes
    V->>S: Registrar venta
    S->>API: POST /sales (items)
    API->>DB: INSERT sales + sale_items
    DB->>DB: TRIGGER restar_stock (valida y descuenta)
    DB->>DB: TRIGGER recalcular_total_venta
    DB->>RT: Evento UPDATE en variants
    RT-->>S: WebSocket
    RT-->>C: WebSocket
    S-->>V: Confirmacion visual
    C-->>C: UI actualizada sin recargar
```

Diagrama: bajo_nivel

```
flowchart LR
    subgraph FE["Frontend Svelte"]
        A1["Login"]
        A2["Catalogo"]
        A3["Detalle"]
        A4["Nuevo/Editar + fotos"]
        A5["Ventas"]
        A6["Reportes"]
        A7["Realtime listener"]
    end
    subgraph BE["Backend Fastify"]
        B1["Auth middleware (JWT)"]
        B2["/products"]
        B3["/variants"]
        B4["/sales"]
        B5["/reports"]
        B6["sharp (imagenes)"]
    end
    subgraph SB["Supabase"]
        C1["PostgreSQL + RLS"]
        C2["Storage"]
    end
```

```

C3["Realtime"]
C4["Auth"]

end
A1 --> C4
A2 --> B2
A3 --> B2
A4 --> B2
A4 --> B6
A5 --> B4
A6 --> B5
A7 --> C3
B2 --> C1
B3 --> C1
B4 --> C1
B5 --> C1
B6 --> C2

```

Diagrama: er

```

erDiagram
    CATEGORIES ||--o{ PRODUCTS : clasifica
    PRODUCTS ||--o{ VARIANTS : tiene
    PRODUCTS ||--o{ PRODUCT_IMAGES : posee
    PRODUCTS ||--o{ PRODUCT_TAGS : etiqueta
    TAGS ||--o{ PRODUCT_TAGS : aplica
    VARIANTS ||--o{ SALE_ITEMS : vendido_en
    SALES ||--o{ SALE_ITEMS : contiene
    PROFILES ||--o{ SALES : registra

    CATEGORIES {
        uuid id PK
        text name
        text slug
        uuid parent_id FK
    }
    PRODUCTS {
        uuid id PK
        text name
        text description
        text image_url
        numeric price
        uuid category_id FK
        boolean active
    }
    VARIANTS {
        uuid id PK
        uuid product_id FK
        text size
        text gender
        text fabric_quality
        text sku UK
        int stock_quantity
    }
    PRODUCT_IMAGES {
        uuid id PK
        uuid product_id FK
        text image_url
        boolean is_primary
        int position
    }
    SALES {
        uuid id PK
        date sale_date
        numeric total_amount
        text customer
        uuid user_id FK
    }
    SALE_ITEMS {
        uuid id PK
        uuid sale_id FK
        uuid variant_id FK
        int quantity
        numeric unit_price
    }
    PROFILES {
        uuid id PK
    }

```

```
text email
text role
}
```

Diagrama: despliegue

```
flowchart LR
    subgraph DEV["Desarrollo local"]
        L1["Vite dev :5173"]
        L2["Fastify dev :3000"]
    end
    subgraph GH["GitHub"]
        R["Repo productos"]
        AC["GitHub Actions"]
    end
    subgraph PROD["Produccion"]
        P1["GitHub Pages (frontend)"]
        P2["Railway (backend)"]
    end
    S1["Supabase Cloud (DB, Storage, Auth, Realtime)"]
    R --> AC
    AC --> P1
    R --> P2
    P1 -- "HTTPS + JWT" --> P2
    P2 --> S1
    P1 -- "Realtime/Storage" --> S1
```

Anexo D. Glosario

Término	Definición
RLS	Row Level Security: control de acceso por fila en PostgreSQL.
Realtime	Actualizaciones en vivo mediante WebSockets.
Trigger	Función que se ejecuta automáticamente ante INSERT/UPDATE/DELETE.
SKU	Código único de la variante (Stock Keeping Unit).
Service key	Clave con permisos de administrador, solo en el backend.
Anon key	Clave pública que usa el frontend, limitada por RLS.
pg_cron	Extensión de PostgreSQL para tareas programadas (Fase 6).
JWT	Token firmado que identifica al usuario autenticado.

Anexo E. Repositorio y recursos

- Repositorio: <https://github.com/disfracesrosaelvira/productos>
- Aplicación (GitHub Pages): <https://disfracesrosaelvira.github.io/productos/>
- Backend (Railway): <https://productos-production-8443.up.railway.app>
- Diagramas interactivos: DiagramasArquitecturaProyecto.html (incluido en el proyecto).
- Mockups interactivos: MockupInteractivoHTML.html (incluido en el proyecto).